

Miniaudio

- config/init pattern
 - create config object, call `init()` routine
 - C library → bare pointers
 - carefull management needed – resource manager
- author of the library: David Reid (mackron)
 - „You need a separate *ma_sound* object for every instance of the sound you want to play simultaneously. The resource manager will ensure the sound data is only loaded once, but keep in mind the reference counting system that automatically unloads sounds (section 6.2.2 in the documentation):
<https://miniaud.io/docs/manual/#ResourceManagement>
A common technique is to load all of your sounds initially at load time and store a cache of *ma_sound* objects. Then whenever you need to play a sound, use *ma_sound_init_copy()*“
- callback will be fired when the sound reaches the end
 - can be used to automatic resource management

```
void my_end_callback(void* pUserData, ma_sound* pSound)
```

Miniaudio – resources

```
#include "miniaudio.h"

class AudioManager {
...
private:
    ma_engine engine;

    // owner of all loaded sounds, with custom deleter (function, that deactivates and deallocates sound)
    std::unordered_map<std::string, std::unique_ptr<ma_sound, void(*)(ma_sound *)>> sound_bank;

...
};

void AudioPlayer::load(const std::string & name, const std::filesystem::path & filename) {

    // sound with custom deleter (function, that deactivates and deallocates sound)
    std::unique_ptr<ma_sound, void(*)(ma_sound *)> new_snd(new ma_sound, [](ma_sound* pSnd) { ma_sound_uninit(pSnd); delete pSnd; });

    if (ma_sound_init_from_file(&engine, name.c_str(), MA_SOUND_FLAG_ASYNC, nullptr, nullptr, new_snd.get()) != MA_SUCCESS) {
        std::cerr << "Failed to load sound:" << name << std::endl;
        delete new_snd;
    }
    else {
        // set some sound parameters...
        ma_sound_set_min_distance(new_snd.get(), 0.5f);
        ma_sound_set_max_distance(new_snd.get(), 100.0f);
        ma_sound_set_volume(new_snd.get(), 2.0f);

        // move the sound into the bank: owned by std::unique_ptr => MUST move (non-copyable)
        sound_bank.emplace("Explosion", std::move(new_snd));
    }
}
```

Miniaudio – play 3D sound

```
#include "miniaudio.h"

class AudioManager {
...
private:
    ma_engine engine;
    // owner of all loaded sounds
    std::unordered_map<std::string, std::unique_ptr<ma_sound, void(*)(ma_sound *)>> sound_bank;

    std::unordered_set<ma_sound*> active_sounds;
    static void my_end_callback(void* pUserData, ma_sound* pSound);
...
};

void AudioPlayer::my_end_callback(void* pUserData, ma_sound* pSound) {
    auto t = static_cast<AudioPlayer*>(pUserData); // get current instance
    t->active_sounds.erase(pSound);                // remove from active
    ma_sound_uninit(pSound);                        // deallocate
}

bool AudioManager::play3D(const std::string& name, float soundX, float soundY, float soundZ, float listX, float listY, float listZ, float
listXDir, float listYDir, float listZDir) {

    ma_sound* original = sound_bank.at("name").get(); // get raw pointer from smart-pointer
    ma_sound* copy_snd = new ma_sound;               // will be dealloc. by callback

    if (ma_sound_init_copy(&engine, original, MA_SOUND_FLAG_ASYNC, nullptr, copy_snd) != MA_SUCCESS) {
        std::cerr << "Failed to copy sound: " << name << std::endl;
        delete copy_snd;
        return false;
    }

    copy_snd->endCallback = my_end_callback; // set callback for automated deletion
    copy_snd->pEndCallbackUserData = this;   // pointer to AudioManager instance (similar to GLFW user pointer)

    ma_sound_seek_to_pcm_frame(copy_snd, 0); // reset to beginning
    ma_sound_set_position(copy_snd, soundX, soundY, soundZ); // set the sound properties
    setListenerPosition(listX, listY, listZ, listXDir, listYDir, listZDir); // set listener position

    // play the sound
    if (ma_sound_start(copy_snd) != MA_SUCCESS) {
        std::cerr << "Failed to play sound: " << name << std::endl;
        return false;
    }

    // remember as active; will be automatically cleared from active when callback fires
    active_sounds.insert(copy_snd);
    return true;
}
```